

ObjGen: Constructing Objects with Digital Genetic Information

Hanna Matusik
MIT CSAIL
Cambridge, Massachusetts, USA
hania@csail.mit.edu

Mina Konakovic Lukovic
MIT CSAIL
Cambridge, Massachusetts, USA
minakl@mit.edu

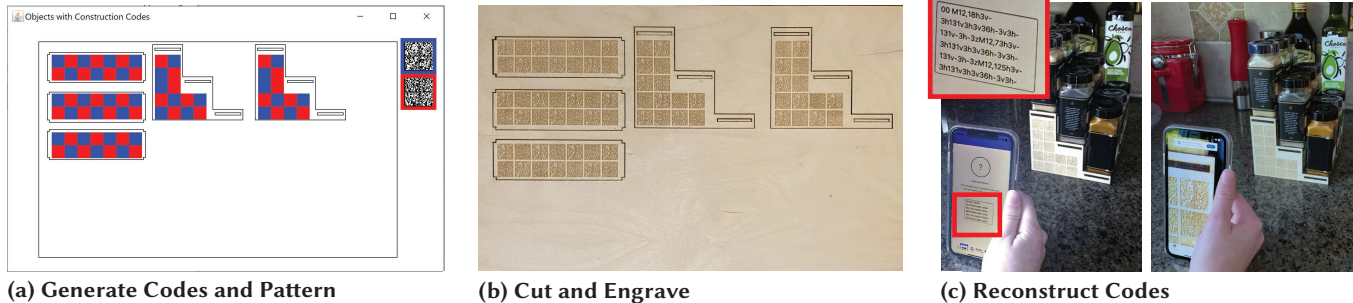


Figure 1: Inspired by genetic information in living organisms, our method encodes the information about the form and functionality of manufactured objects onto their surface. (a) The code for the input SVG model is encoded using Data Matrix cells, and the cells are distributed over the object. (b) The object is cut (or CNC milled), and the code is engraved using a digital cutter. (c) The object’s construction code can be decoded using a cell phone camera.

ABSTRACT

The genome of each living organism encodes its phenotype. Inspired by this phenomenon, we have designed a new method that allows embedding digital genetic information in everyday objects, which we call the ObjGen. These embeddings encode detailed information on how each object is built as well as additional metadata (e.g., manufacturing instructions, assembly instructions, or user’s manual). By scanning only a small fragment of an object, the whole object can be recreated and its metadata read. We show how this workflow can be implemented for different manufacturing processes: laser cutting, CNC milling, and molding. Our method creates digital genetic codes in the form of digital textures incorporated into the object’s surface. The embedded construction code can be decoded using standard digital cameras. We demonstrate this workflow on a variety of different objects.

CCS CONCEPTS

• **Human-centered computing**; • **Human computer interaction (HCI)**; • **Interactive systems and tools**;

KEYWORDS

fabrication; manufacturing; making; laser cutting; identification; tags

ACM Reference Format:

Hanna Matusik and Mina Konakovic Lukovic. 2023. ObjGen: Constructing Objects with Digital Genetic Information. In *Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems (CHI EA '23)*, April 23–28, 2023, Hamburg, Germany. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3544549.3585781>

1 INTRODUCTION

All living organisms, plants and animals, store genetic information in their cells. This genetic information describes how each of the organisms looks and functions. Using a small fragment of even a single cell organism, one could sequence its genome. This information provides a method to recreate this organism. In contrast, man-made structures and objects typically do not have these properties. Once the object is damaged, it cannot be easily recreated. Digital manufacturing allows for computer representations (for example, CAD models) to be automatically translated to manufactured objects. However, the digital representation is stored separately from that object. This representation can be lost, destroyed, or not easily accessible. In this work, we present a new method that uses standard digital manufacturing equipment, in our case, digital laser cutters, CNC machines, or molding equipment, to imbue an object with an equivalent of genetic information. We take advantage of the feature that digital cutters and CNC machines can both cut out an object and engrave patterns on it. In our method, the instructions on how to make the object are converted to a texture pattern that is engraved on the object’s surface. This texture pattern can be scanned using digital cameras and converted back to digital instructions. We develop a complete encoder-decoder architecture for this process. A Data Matrix [32] code is used to support variable cell size and error correction. We demonstrate our method on a variety of man-made objects. Our framework is evaluated in terms of code size detection accuracy and manufacturing resolution. We envision the following

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
CHI EA '23, April 23–28, 2023, Hamburg, Germany
© 2023 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9422-2/23/04.
<https://doi.org/10.1145/3544549.3585781>

use cases for objects with digital genetic information. First, access to digital files in the cloud might not be available if the manufacturer goes out of business, the server is destroyed, or in case of natural disasters (e.g., floods, earthquakes), military conflicts, cyberattacks, or deliberate blocking of the Internet by different countries. In such cases, even damaged objects with digital genetic information can still be recreated or replicated. Using a simple digital phone application, a user can read a complete digital manufacturing instruction from a fragment of the part and then replicate the part or the entire object. Unlike when the information about the object is stored on a server, the genome code is accessible for as long as at least one piece of an object exists. Second, additional metadata can be stored with the object. For example, if the object requires step-by-step assembly instructions, these instructions can be included with the object. Similarly, the user's manual can be added to the genetic information. Furthermore, consider a digital drone that requires control software, and that software is accidentally erased. That software can be kept directly encoded in the drone's body and recreated if it is erased. Our method can be conveniently applied in all the situations listed above. To summarize, the contributions of this work are as follows: 1) We introduce a method that integrates the information about the object's structure and functionality into its physical form and reads this information without destroying the object. 2) We provide a design tool that performs the coding and decoding of an object's information and the distribution of the code on a given object. 3) We demonstrate the whole process on a variety of everyday objects.

2 RELATED WORK

Barcodes. Barcodes have been developed to embed digital information in the form of visual patterns. They can be printed or placed on many different objects. Typically, the information can be reliably scanned or retrieved using special optical scanners or even simple cell phone cameras. One-dimensional barcodes encode a limited amount of information in a pattern of black and white stripes of varied thicknesses. 2D barcodes work similarly, but they can store significantly more information. The two most popular encoding schemes are QR codes [34] and Data Matrix codes [32]. Within each code standard, we can change the size of the pattern, which determines the number of encoded bits. With these standards, up to 2000 characters can be encoded in a single code. In addition to this, the codes also use error correction. The error correction means up to 20% of the bits can be corrupted, and the code can still be read reliably. We take advantage of two-dimensional Data Matrix codes as a component of our system. More recently, Tancik et al. [28] demonstrated how to imperceptibly encode information into regular photographs using deep learning encoders. Information can be robustly decoded by taking a picture with a regular cellphone camera. The advantage of this approach lies in its ability to use any texture as a carrier of the information which makes it more aesthetically pleasing. Unfortunately, only about 100 bits can be encoded this way. This size is typically not enough information to encode the whole digital object. Similarly, Chu et al. [4] create QR codes that are more visually pleasing. Arbitrary image patterns can be included as the basis of the code.

Codes in Digital Fabrication. Many digital fabrication techniques allow embedding data inside of physical objects. A comprehensive overview of the methods in the context of additive manufacturing has been presented by Hou et al. [15]. The information can be stored in a few different ways. First, the information can be stored by changing the appearance of the manufactured objects. In this category, Maia et al. [23] allow encoding barcodes in 3D printed objects by altering printed layers. Two variants of this technique are presented: layers with different colors and layers with different IR materials. A similar approach is employed by Kennedy et al. [18]. Getschmann and Ehtler [12] study the aesthetics of markers for tagging physical objects and introduce more aesthetically pleasing markers based on Voronoi diagrams. In the second category, the object's external geometry is modified. For example, the codes can be embossed on the surface of the object either as QR codes [19] or as less visible noise patterns [2]. We employ similar methods using laser cutting, CNC machining and molding. The last category is patterns that are placed inside the objects. For example, Willis and Wilson [35] create InfraStructs – physical tags embedded into physical objects that can be read with terahertz imaging methods. Similarly, Li et al. [21] employ empty air pockets inside 3D printed objects that are invisible under natural lighting. Information can be decoded using a projector and camera system. Chen et al. [3] disperse the code throughout the 3D object and use a micro-CT scanner to read it. Other approaches use IR imaging [9, 25]. Dogan et al. [8] develop a method that explores different slicing parameters in 3D printing. Information is encoded in surface and infill patterns. All these approaches allow adding almost imperceptible features to digitally manufactured objects, and it is easy to change this information for each object. However, the amount of the information is very limited, and usually, only tens of bytes are stored. There is usually no data redundancy in all these approaches – only one code is embedded in the object. Therefore, if an object is damaged or a part of it is missing, the encoded information could be impossible to retrieve. In our approach, we need as little as 3% of the object to recreate it and retrieve its associated metadata. Recently, Ettehadi et al. [11] proposed a workflow to embed and retrieve tags in 3D printed objects with the purpose of incorporating spatial documentation information. They find the embedded information useful. However, the aesthetics of the tags is an important consideration for the users. In our approach, we also allow incorporating metadata into an object. However, the codes are incorporated throughout most of the object's surface and are generally less intrusive. Finally, Koch et al. [20] have proposed a method to construct custom DNA sequences that encode high-density information about the structure of a digital object. These DNA sequences are embedded into polymeric materials and used as filaments to 3D print this object. Fragments of the 3D printed object can be analyzed, and the corresponding information can be decoded using a DNA sequencer. They demonstrate impressive results of encoding 45 KB Stanford Bunny in a 3D printing filament. The major downside of this method is that a new DNA sequence and filament need to be manufactured for each new object, which makes it impractical for many applications. Furthermore, the object needs to be destroyed in order to read the DNA sequence. Finally, reading the object's code requires expensive equipment and a lengthy process that is out of reach for typical users.

3 METHOD

Workflow Overview. We present a method for embedding construction codes into man-made objects. Our method uses a standard digital laser cutter or a CNC milling machine to engrave the construction code and to cut the object. Alternatively, an inverse mold can be cut and engraved on the same machines. In this method, we do not need to destroy the object in order to read its construction code. We develop a design tool that allows users to load and display objects for manufacturing. Since our manufacturing technique is laser cutting or CNC milling, the objects are loaded as sets of 2D geometric models that are assembled together after cutting. The 2D models are loaded as vector files, e.g., SVG files. The user can then choose both the physical size of the cell and the code size – the larger the code, the more information it can store. The user can also specify metadata associated with the file. The metadata can be directly entered as text in the design tool. Furthermore, the user can select whether to encode the data in ASCII or binary compressed format. Information can also be encrypted and password protected. The program then generates a code pattern for engraving. This pattern populates the object with cells. Each cell contains a part of the object’s information. Each part of the object contains the information of the entire object. Both the pattern and the original file are loaded to a digital cutter or CNC milling machine. They are aligned, and the object is then cut and engraved. Finally, the object can be assembled from its parts. If users wish to recreate the object, retrieve their digital form, or read the metadata, they simply have to scan the codes using a smartphone and utilize our tool to decode the information. The workflow is summarized in Figure 1.

Input. Digital cutters and CNC milling machines take input in the form of vector files [13, 27, 33]. We have designed a more compact version of an SVG file format with a limited set of commands but without sacrificing its expressiveness. Our file format supports the following commands: M - moves to a point, V - draws a vertical line, H - draws a horizontal line, L - draws a line, A - draws an arc, C - draws a curve, Z - closes the contour. The format works for both commands in absolute coordinates (capital letters) and relative coordinates (small letters). This small set of commands allows encoding the input more efficiently. Users can select the data to be stored in three different formats: ASCII, binary compressed, and binary compressed and encrypted format. The desired format can be selected by the user. Our tool is combined with the 7-Zip [1], a free and open-source file archiver, to support data compression and encryption. To obtain a good compression of the SVG data, we use prediction by partial matching (PPM) [5] supported by 7-Zip. We use Advanced Encryption Standard (AES-256) [10] for data encryption supported by 7-Zip. Users can enter the desired password directly in our design tool. Our input format also allows us to store metadata along with the SVG file. The metadata can be directly entered as text in our tool. If data is stored in the ASCII format, the metadata is added at the end of the SVG file. However, if the data is compressed, the metadata is stored in a separate text file in the compressed file archive.

Encoder. The input data is converted into a 2D image that can later be engraved. This image is based on two-dimensional codes. There are a few popular types of 2D codes: QR Code, Data Matrix,

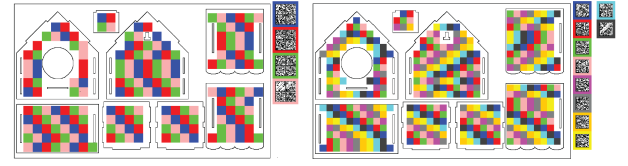


Figure 2: Bird House model encoded using two different cell sizes. Left: Size 40 Data Matrix at 18 mm. This size requires 4 different cells depicted in different colors to encode the whole object. Right: Size 26 Data Matrix at 13 mm. The smaller size requires 10 different cells for the Bird House object.

Aztec Code [6]. We chose Data Matrix since it is the most visually pleasing. The code resembles a random pattern and does not have any significant features. A 10x10 code can store one byte, while a 144x144 code can store 1555 bytes. If we need to encode more than 1555 bytes, we use multiple Data Matrix codes, as discussed next. All Data Matrix codes have error correction. We use a Java library ZXing [36] to encode SVG files in our design tool.

Generating Texture Patterns. The use of Data Matrix codes for encoding data limits how much information can be stored in one cell. Usually, the amount of information necessary to encode the whole object is larger than the capacity of one cell. Another consideration is that it is better to keep the cell size small. For example, if the object has lots of thin surfaces rather than large flat areas, it may not be possible to fit large cells inside the 2D object. To fix the sizing issue, the user can select the desired cell size appropriate for the geometry of the object (Figure 2). The necessary information is then split across multiple cells. For example, if the information required to encode the object is 1024 bytes, and each cell can store 256 bytes, four different cells with codes are used, each representing a part of the object’s code. In addition, we inserted a small header as the first three bytes in each cell to denote the code part number. The last cell stores a special character sequence (an extra “.” character after the number) to denote the end of the file. We generate the engraving pattern by automatically tiling different code cells. Distributing the codes over all cells ensures that the entire code can be retrieved even after some object parts are damaged or lost. We note that using compressed files results in a much shorter program, and therefore fewer code cells are required (Figure 3). There are a few different ways to distribute codes into cells. The distribution can be done using 1) a random pattern, 2) a regular pattern, or 3) an optimized pattern. For a random pattern, each cell is assigned a random code number. For a regular pattern, all the cells are counted and assigned a number. Then, the code for each cell is its cell number modulo the number of different codes. The code cells are placed on the object in a row-by-row manner. After placing all the different cells, they are repeated until the whole object is covered. However, we obtain the best results with an optimized assignment of codes to cells. The general insight is that we would like to have all codes within each spatial neighborhood. This feature is very useful when decoding the object’s information since we only need to scan codes in a small spatial neighborhood to assemble all information. The optimization process starts by assigning the codes to cells randomly. Then, the process iterates over all cells. For each cell, we compute



Figure 3: The same object encoded with 7-Zip compression requires 12 different code cells (left) and a non-compressed text file requires 42 different code cells (right).

the distances to the closest cell with each code. The code that has the furthest distance is assigned to the current cell. This process converges quickly within a few seconds.

Manufacturing. We investigated laser cutting to manufacture objects. We use the same digital cutter to both cut and engrave (Dremel Digilab LC40 [7]). We typically first engrave the pattern and then align the object’s cutting pattern with the engraved material. Engraving before cutting is advised, especially for not perfectly flat materials and may slightly move after the cutting process. This digital cutting machine also has a camera that makes this alignment process much easier to do. We usually keep the whole sheet of material in the same place and do not remove the non-object part of the material until the cutting is completed. Alternatively, we can use a similar setup to manufacture a mold for casting the object. The advantage of molding is that multiple copies of the same object can be rapidly manufactured, which amortizes the fabrication time for the mold. Furthermore, a possible material set can be expanded to include epoxy, silicone, and urethane. Typically, the mold consists of two pieces: the bottom part is a flat sheet that is engraved with the negative of the texture pattern; the top part is a laser cut inverse shape (example mold is shown in Figure 9, fifth row, center column). For our experiments, we use urethane resins (Smooth Cast 305 urethane resin) with 30 minutes curing time to produce rigid parts. We also use a mold release spray to ensure the object can be easily separated from the mold. We also demonstrate the same workflow with CNC milling as a manufacturing method. CNC milling expands the range of materials that can be used to manufacture objects. In particular, we show an example of a metal object. We use an Southwestern Industries Prototrak milling machine and employ one drill bit for milling the contour and one for engraving. We show an example CNC milled object in Figure 9 (second row). Overall, we have tested different types of materials: plywood sheets, balsa sheets, and plastic sheets of different colors (black, white, and clear), casting resins, and metals. In general, laser cutting and engraving works well for wood and plastics, while CNC machining can support a wider range of materials including metals. As a part of our analysis, we also determine the maximum possible Data Matrix resolution size of the engraving process such that the pattern is clearly visible and detectable. We have tested the resolution based on the Data Matrix codes (size 40) ranging from 7 mm to 20 mm millimeters. We determined that 18 mm is the pattern size that can still be clearly visible and detectable (Figure 4). We have observed that when a few “black” pixels are engraved next to each other, the pixels become less uniform in the shade (see the effect as the horizontal line in the middle of the pattern in Figure 5). To improve the detection, we replace each black pixel in a Data Matrix with a 4x4 checkered



Figure 4: The same Data Matrix code printed with different resolution. Codes highlighted red are not detectable. Codes highlighted green are detectable.

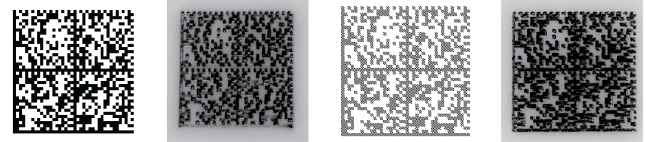


Figure 5: From left to right: Data Matrix code (full, digital); corresponding image engraved; Data Matrix code (checkered); corresponding image engraved. The horizontal line in the middle of the full pattern is almost invisible while the same line in the checkered pattern is detectable.

pattern (e.g., [0 1 0 1; 1 0 1 0; 0 1 0 1; 1 0 1 0]). This replacement creates a micro pattern that holds the ink (or burnt wood) in place.

Decoding. The main requirement for the detection process is not to rely on expensive additional equipment and use only a standard smartphone camera. For the Data Matrix decoder, there are various libraries that work directly on smartphones, which makes the decoding process easy. Since each object is usually encoded using multiple code cells, all different codes need to be scanned to reconstruct the object and its metadata. In practice, each code has an ASCII header that stores the code number. The header is visible regardless of whether the rest of the code is stored in an ASCII or a binary format. The last code cell has an extra “.” character after the code number and therefore reveals the number of different code cells the user has to scan. After the user scans at least one code with each code number, the file can be assembled together. Our design tool can be used to read all scanned codes, remove their headers, arrange the codes in proper order, and produce the final file in an ASCII or binary format. The patterns engraved in wood can be directly scanned. However, for other materials, the pattern is barely visible. Our solution is to apply a small amount of pigment to the pattern before it is scanned. We apply black ink to the engraved patterns on materials that are in the light or clear colors. After pigmentation, the codes are visible and detectable for decoding. We apply white pigment to the patterns engraved on dark materials. In this case, we need to invert the picture of the pattern before it can be scanned and decoded. The pigments can be easily cleaned from the pattern after they are scanned (Figure 6).

4 RESULTS

Example Objects. We have designed different objects in Inkscape [16] (a free drawing program for 2D vector graphics) to be tested by the system. All objects are realistic and practical. Some of them are similar to the designs that are found on Thingiverse [29] and

Instructables [17]. However, the digital designs are much more compact because we do not use any additional information that can significantly increase the file size. All examples are shown in Figure 9. **Bird House:** A wooden DIY small bird house that can be hanged or attached in a garden. The design includes eight separate pieces that can be cut from a single piece of wood. It can be assembled together using snap fits without any additional screws or adhesives. The entire design is 886 bytes, the compressed design is 401 bytes, and the encrypted, password-protected file is 434 bytes. **Candle Holder:** A design for a candle holder composed of four pieces. The design can be assembled by putting the four parts together. The design is 297 bytes in uncompressed format, 246 bytes in compressed format, and 282 bytes in encrypted, password-protected format. We manufacture the design both in wood using a laser cutter and in aluminum using CNC milling. **Stackable Box:** A design for a small crate that can be stacked on top of one another. The edges are rounded using curves, and it has comfortable handles. It is made up of five pieces and can be assembled using snap fits as well. The design is 1466 bytes in uncompressed format, 495 bytes in compressed format, and 538 bytes in encrypted, password-protected format. Boxes are made of wood, black plastic, and white plastic. **Fruit Bowl:** A design for a fruit bowl made from an interlocking pattern of circles. This example is the most complex object out of all the other objects evaluated. It uses 11 separate pieces that are stored using 1999 bytes in uncompressed format, 517 bytes in compressed format, and 554 bytes in encrypted, password-protected format. **Paper Towel Stand:** A stand that can hold a regular sized eight-inch paper towel (e.g., to be used in a kitchen). The design is made from three pieces and can be assembled without any glue. The uncompressed design is 481 bytes, the compressed design is 308 bytes, and the encrypted, password-protected design is 346 bytes. We manufacture the design using wood and also by casting it using an urethane resin. **Drone:** We use the components from a small do-it-yourself drone kit (ZL100 DIY Drone). We have redesigned the drone's body. The drone's body is made using five pieces. The design in uncompressed format is 1071 bytes, 416 bytes in compressed format, and 458 bytes in encrypted, password-protected format. The model can be made from 1.5 mm thick balsa wood sheets. This design also requires additional components: controlling board, battery, four motors, and four propellers. **Fluidic Mixer:** Fluidic mixers are commonly used in many laboratory applications. We use a design of a simple mixer made from four stacked acrylic pieces [26]. The mixer takes two inlets with different fluids and mixes both fluids. The mixed fluid is let out in the outlet. The design is 1507 bytes in uncompressed form, 444 bytes in compressed form, and 490 bytes in encrypted, password-protected format. The mixer is laser cut from clear acrylic sheets.

Metadata. In addition to storing object's shape in the code, additional metadata information that provide a more comprehensive description of the object can be added. We illustrate this with two examples of designed and manufactured objects (Figure 7). First, metadata can include the details of the manufacturing process. For example, it is often useful to describe the type of material used for manufacturing, machine type, and its detailed settings for cutting and engraving. It might be beneficial to provide this information

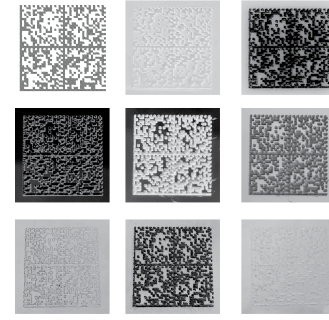


Figure 6: Testing our method on different types of materials. First row, from left to right: A (checkered) pattern to be engraved; the same pattern engraved in light acrylic plastic is almost invisible; the same pattern with applied black pigment is visible and can be scanned. Second row: The pattern engraved in dark acrylic plastic is not very visible; the same pattern with applied white pigment; the inverted image of the previous pattern can be easily scanned. Third row: The pattern engraved in transparent plastic is almost invisible; the same pattern with applied black pigment can be easily scanned; the pigment can be easily cleaned after scanning (last image).

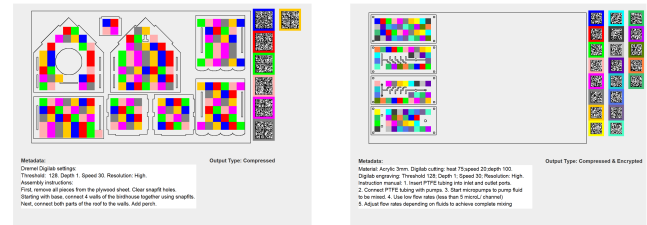


Figure 7: Examples of different types of metadata that are stored in fabricated objects: desired manufacturing equipment and its settings, desired material type, assembly instructions, and instruction manual.

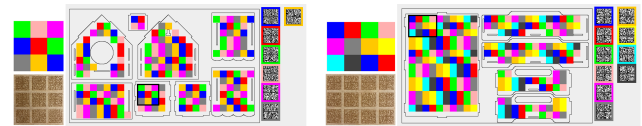


Figure 8: Examples of reconstructing a complete construction code from a small fragment of the original object. Bird House model is correctly reconstructed from a small square area containing nine cells and seven unique codes (left). Stackable Box model is correctly reconstructed from a rectangular area containing twelve cells and ten unique codes (right).

not just for one equipment and material type, but also for various types. Second, assembly instructions can be incorporated in metadata as well. These instructions are especially useful when the objects are more complex and have many pieces. For example,

we demonstrate this idea with assembly instructions for our bird house model, which tell the user in which order to assemble the components. Third, it is often beneficial to add an instruction manual to the manufactured object. In this case, users will always have access to the manual even when the object is assembled. These instructions can be beneficial when the instruction manual is lost or when access to the material on the Internet is not available. We demonstrate this concept by including an instruction manual for one of our examples – the fluidic mixer. Finally, it may be useful to add a hyperlink that stores extensive information about the object available on the Internet or another type of identifier for the object.

Evaluation. We focus our evaluation on examining the decoding of the engraved patterns – a critical component of the workflow. We have evaluated a few different libraries/applications that work on smartphones for scanning the codes. Two of them – QR Reader for iPhone [22] and QR Code Reader [30] – work approximately the same. Both of them decode QR and Data Matrix codes. The fact that both are interactive (the detector for Data Matrix codes works directly on the captured video) makes the detection process easier. Uniform lighting makes the detection more reliable. For our code scanning experiment, we have evaluated the detector on four different smartphones, each using a different image sensor: iPhone 7, iPhone 11, iPhone 12, and Samsung Galaxy S22 Ultra. Using each of the phones, we scanned 100 different Data Matrix codes. Each of the codes could be read correctly on every phone. This observation suggests the high reliability of the proposed method. We have also evaluated the whole process by reconstructing complete files of several objects (Figure 8). For this task, we only use a small fragment of the physical surface. Our optimized assignment of codes to cells ensures that each code is present in a small spatial neighborhood. On the contrary, random assignment of codes to cells does not have this guarantee. Similarly, regular pattern might result in suboptimal assignment. For example, the Bird House model is encoded using seven construction codes, and a 3x3 fragment is enough to reconstruct the model. Similarly, the Stackable Box uses ten construction codes, and therefore 4x3 fragment lets us reconstruct the original model. These results are not surprising as the scanning process is robust.

Discussion. **Data density:** The amount of information that can be embedded by engraving an object is limited to about 43 bytes/cm² – one of the main constraints of the method. Microfabrication methods could significantly increase data density. Incorporating codes within the whole volume rather than just placing them on the surface could give further improvements. **Code Redundancy:** In comparison to most previous approaches, where only a single code is embedded, our approach provides a large code redundancy. The redundancy allows us to reconstruct an entire object when as little as 3% of the object is available (calculated as a fraction of object's surface that contains a square patch with all codes). **Effect of the cell size:** In our method, the cell size is one of the main parameters. When the cells are smaller, one can better fit them to the geometry of the object. However, one would need more cells to fit all the information of the object. On the other hand, when the objects are bigger or when they have very large flat surfaces (e.g., in the bird house or the stackable box), one can use bigger cell sizes. Currently, our design tool allows users to select both the physical cell size and

the Data Matrix size. **Appearance and aesthetics:** The patterns from the Data Matrix codes resemble random patterns. However, decoders require a quiet zone between each of the cells. This requirement makes the objects look less aesthetically pleasing, especially for the objects made of wood. In general, for many objects, one can place the pattern only on one side that is usually invisible. For example, the pattern can be placed inside of the bird house. We also note that the aesthetics of the pattern largely depends on the material being used. For example, the patterns are clearly visible for wood, and they are almost invisible when using plastic materials (see Figure 6). **Mechanical properties and surface texture:** We observed that the addition of the engraving pattern does not have any significant impact on the mechanical properties of the object – the engraving depth is relatively low compared to the thickness of the object. We keep the engraving depth to 0.2 mm for wood and 0.1 mm for plastics and metal compared to the typical 3 mm thickness of the material sheet. We acknowledge that the mechanical properties might decrease when working with very thin material sheets. The surface texture can also be undesirable in some areas of the object. In the current process, the user can edit the texture image before it is being engraved and remove the codes in the areas where they might conflict with the proper function of the object. As an alternative, we plan to add a feature in our application to allow specifying restricted areas directly on top of the SVG file. **Intellectual property:** Allowing unrestricted access to CAD files can be potentially unacceptable for many designers and manufacturers. We mitigate this issue by allowing encryption and password protection of the stored information. We use the encryption and password protection available in the 7-Zip library. More sophisticated data encryption schemes can also be used. We note that it might be beneficial to encrypt only a part of the information. For example, a designer might want to encrypt and password-protect the SVG file and leave the metadata without password protection. **Engraving time:** We note that the engraving process can increase the total manufacturing time. Our method is not practical when the only objective is to maximize the throughput of the manufacturing process and when objects are directly cut. However, when objects are molded, the additional time required for mold generation is amortized over many replicas of the object. In many cases incorporating construction information and metadata can enhance the utility of objects and outweigh this additional cost. **Embedding construction codes vs. storing hyperlinks:** One possible alternative to storing a full construction code and its metadata in a physical object is to store a link to a website where the construction information and the metadata is stored. While storing a hyperlink requires much less information (and it could be also done with our approach), it also has significant disadvantages. First, some countries (e.g., China [14], and more recently Russia [24]) block access to Internet. In some other instances, specific websites can be taken down by cyberattacks[31]. Furthermore, Internet can be simply inaccessible during military conflicts and natural disasters. Finally, websites and the associated information can be simply taken down (e.g., when companies go out of business).

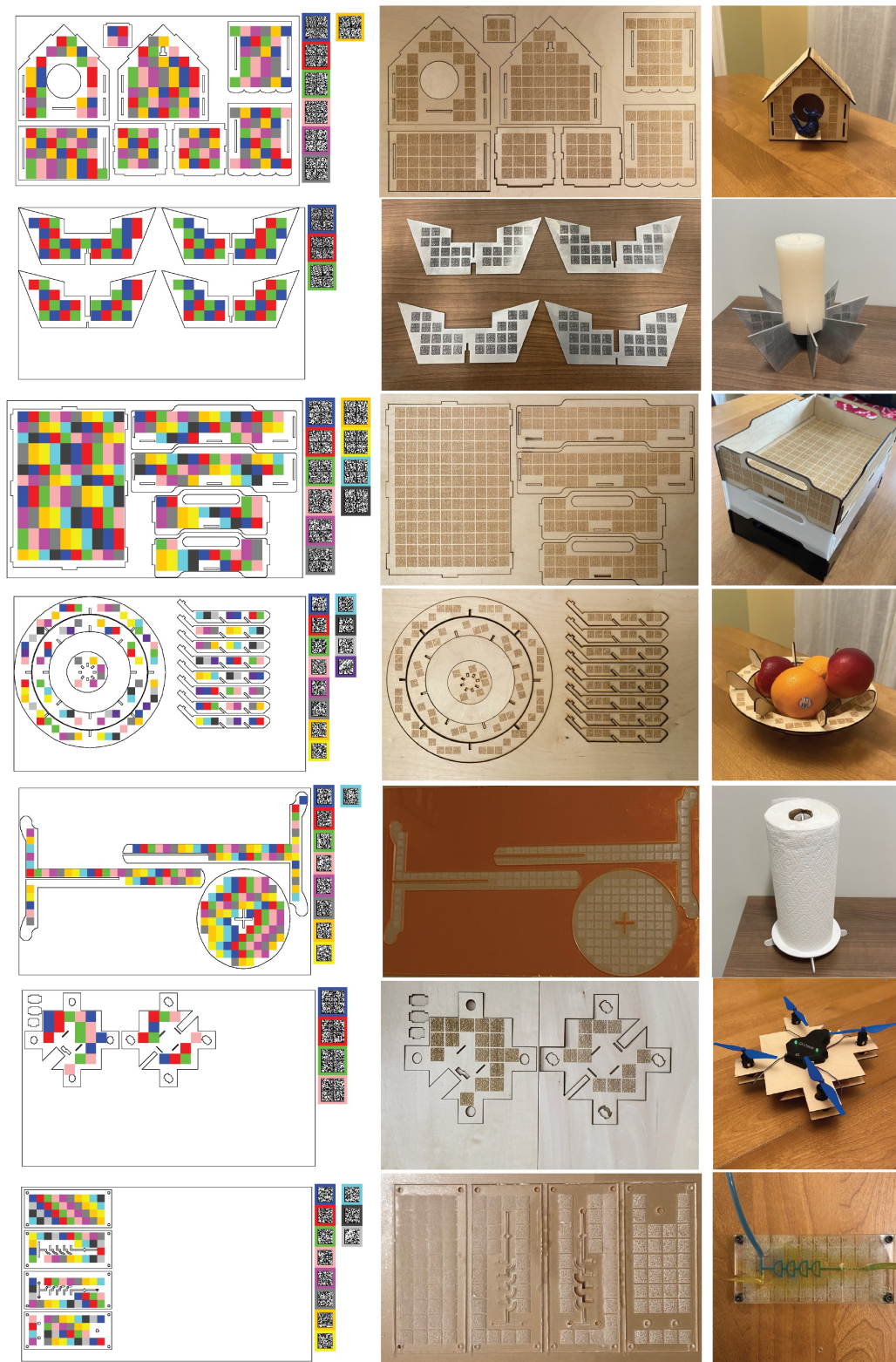


Figure 9: Results: Model with code cells (different cells are color-coded) and code types on the side (left column), cut and engraved objects (center column), assembled objects (right column).

5 CONCLUSION AND FUTURE WORK

In this project, we have proposed to modify a digital manufacturing workflow for digital cutting, CNC machining, and molding. Taking inspiration from nature, we developed a method that allows one to enhance physical objects with additional information about their structure and functionality. The manufacturing instructions are converted into a pattern that is engraved on the surface of the object. These patterns can be non-destructively decoded from a small part of an object, and the object could be conveniently recreated from the decoded information. A few components of our method could be improved in the future. First, the current engraving patterns can be aesthetically unpleasing. We could fix this problem with codes that form natural patterns. The second problem of our method is the limited amount of information that can be encoded. The cell sizes tend to be relatively large. This limitation could be improved by using more precise methods of engraving. Our work has many possible avenues for future work. First, we would like to extend our approach to other manufacturing methods. This extension would automatically expand the range of materials from which the objects can be manufactured, such as glass, fabrics, and natural fibers.

REFERENCES

- [1] 7-Zip. 2021. 7-Zip. 7-Zip. <https://www.7-zip.org/>
- [2] Daniel G. Aliaga and Mikhail J. Atallah. 2009. Genuinity Signatures: Designing Signatures for Verifying 3D Object Genuinity. *Computer Graphics Forum* 28, 2 (2009), 437–446. <https://doi.org/10.1111/j.1467-8659.2009.01383.x> arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1111/j.1467-8659.2009.01383.x>
- [3] Fei Chen, Yuxi Luo, Nektarios Georgios Tsoutsos, Michail Maniatakos, Khaled Shahin, and Nikhil Gupta. 2019. Embedding Tracking Codes in Additive Manufactured Parts for Product Authentication. *Advanced Engineering Materials* 21, 4 (2019), 1800495. <https://doi.org/10.1002/adem.201800495> arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1002/adem.201800495>
- [4] Hung-Kuo Chu, Chia-Sheng Chang, Ruen-Rone Lee, and Niloy Mitra. 2013. Halftone QR codes. *ACM Trans. Graph.* 32 (11 2013), 217:1–217:8. <https://doi.org/10.1145/2508363.2508408>
- [5] John Cleary and H. Witten. 2003. Data Compression Using Adaptive Coding and Partial String Matching. *IEEE Transactions on Communications* 32 (03 2003). <https://doi.org/10.1109/TCOM.1984.1096090>
- [6] Aztec Code. 2021. *Aztec Code Specification*. ISO. <https://www.iso.org/standard/41548.html>
- [7] Dremel Digilab. 2021. Dremel Digilab LC40. <https://digilab.dremel.com/products/lc40-laser-cutter>. [Online; accessed 10-September-2021].
- [8] Mustafa Doga Dogan, Faraz Faruqi, Andrew Day Churchill, Kenneth Friedman, Leon Cheng, Sriram Subramanian, and Stefanie Mueller. 2020. *G-ID: Identifying 3D Prints Using Slicing Parameters*. Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3313831.3376202>
- [9] Mustafa Doga Dogan, Ahmad Taka, Michael Lu and Yunyi Zhu, Akshat Kumar, Aakar Gupta, and Stefanie Mueller. 2022. *InfraredTags: Invisible AR Markers and Barcodes Using Low-Cost, Infrared-Based 3D Printing and Imaging Tools*. Association for Computing Machinery, New York, NY, USA, 1–12.
- [10] Morris Dworkin, Elaine Barker, James Nechvatal, James Foti, Lawrence Bassham, E. Roback, and James Dray. 2001. Advanced Encryption Standard (AES). <https://doi.org/10.6028/NIST.FIPS.197>
- [11] Omid Ettehadi, Fraser Anderson, Adam Tindale, and Sowmya Somanath. 2021. Documented: Embedding Information onto and Retrieving Information from 3D Printed Objects. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 424, 11 pages. <https://doi.org/10.1145/3411764.3445551>
- [12] Christopher Getschmann and Florian Echtler. 2021. Seedmarkers: Embeddable Markers for Physical Objects. In *Proceedings of the Fifteenth International Conference on Tangible, Embedded, and Embodied Interaction (Salzburg, Austria) (TEI '21)*. Association for Computing Machinery, New York, NY, USA, Article 26, 11 pages. <https://doi.org/10.1145/3430524.3440645>
- [13] Scalable Vector Graphics. 2018. Scalable Vector Graphics – W3C Candidate Recommendation. <https://www.w3.org/TR/SVG2/>. [Online; accessed 10-September-2021].
- [14] Steve Guo and Guangchao Feng. 2012. Understanding Support for Internet Censorship in China: An Elaboration of the Theory of Reasoned Action. *Journal of Chinese Political Science* 17, 1 (2012), 33–52.
- [15] Jong-Uk Hou, Dongkyu Kim, Wonhyuk Ahn, and Heung-Kyu Lee. 2018. Copyright Protections of Digital Content in the Age of 3D Printer: Emerging Issues and Survey. *IEEE Access* 6 (2018), 44082–44093.
- [16] Inkscape. 2021. Inkscape. <https://inkscape.org/>. [Online; accessed 10-September-2021].
- [17] Instructables. 2021. Instructables. <https://www.instructables.com/>. [Online; accessed 10-September-2021].
- [18] Z. C. Kennedy, D. E. Stephenson, J. F. Christ, T. R. Pope, B. W. Arey, C. A. Barrett, and M. G. Warner. 2017. Enhanced anti-counterfeiting measures for additive manufacturing: coupling lanthanide nanomaterial chemical signatures with blockchain technology. *J. Mater. Chem. C* 5 (2017), 9570–9578. Issue 37. <https://doi.org/10.1039/C7TC03348F>
- [19] Ryosuke Kikuchi, Sora Yoshikawa, Pradeep Jayaraman, Jianmin Zheng, and Takashi Maekawa. 2018. Embedding QR codes onto B-spline surfaces for 3D printing. *Computer-Aided Design* 102 (05 2018). <https://doi.org/10.1016/j.cad.2018.04.025>
- [20] Julian Koch, Silvan Gantenbein, Kunal Masania, Wendelin J. Stark, Yaniv Erlich, and Robert N. Grass. 2020. A DNA-of-things storage architecture to create materials with embedded memory. *Nature Biotechnology* 38, 1 (2020), 39–43. <https://doi.org/10.1038/s41587-019-0356-z>
- [21] Dingzeyu Li, Avinash S. Nair, Shree K. Nayar, and Changxi Zheng. 2017. Air-Code: Unobtrusive Physical Tags for Digital Fabrication. In *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology, UIST 2017, Quebec City, QC, Canada, October 22 - 25, 2017*. Association for Computing Machinery, New York, NY, USA, 449–460.
- [22] TapMedia Ltd. 2022. QR Reader for iPhone. <https://apps.apple.com/us/app/qr-reader-for-iphone/id368494609>. [Online; accessed 7-April-2022].
- [23] Henrique Teles Maia, Dingzeyu Li, Yuan Yang, and Changxi Zheng. 2019. Layer-Code: Optical Barcodes for 3D Printed Shapes. *ACM Trans. Graph.* 38, 4, Article 112 (July 2019), 14 pages. <https://doi.org/10.1145/3306346.3322960>
- [24] Adam Satariano and Valerie Hopkins. 2022. Russia, Blocked From the Global Internet, Plunges Into Digital Isolation. <https://www.nytimes.com/2022/03/07/technology/russia-ukraine-internet-isolation.html>. [Online; accessed 7-March-2022].
- [25] Piyyarat Silapasuphakornwong, Masahiro Suzuki, Hiroshi Unno, Hideyuki Torii, Kazutake Uehira, and Youichi Takashima. 2016. Nondestructive readout of copyright information embedded in objects fabricated with 3-D printers. In *Digital Forensics and Watermarking: 14th International Workshop, TWDW 2015, Tokyo, Japan, October 7-10, 2015, Revised Selected Papers 14*. Springer, New York City, 232–238.
- [26] S. Sivashankar, Sumeyra Agambayev, Yousof Mashraei, Erqiang Li, Sigurdur Thoroddsen, and Khaled Salama. 2016. A “twisted” microfluidic mixer suitable for a wide range of flow rate applications. *Biomicrofluidics* 10 (05 2016), 034120. <https://doi.org/10.1063/1.4954812>
- [27] Adobe Systems. 2021. *Adobe Illustrator File Format Specifications*. Adobe Systems. <https://web.archive.org/web/20150906044646/http://partners.adobe.com/public/developer/en/illustrator/sdk/A17FileFormat.pdf>
- [28] Matthew Tancik, Ben Mildenhall, and Ren Ng. 2020. StegaStamp: Invisible Hyperlinks in Physical Photographs. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020*. Computer Vision Foundation / IEEE, New York, 2114–2123. <https://doi.org/10.1109/CVPR42600.2020.00219>
- [29] Thingiverse. 2021. Thingiverse. <https://www.thingiverse.com/>. [Online; accessed 10-September-2021].
- [30] TinyLab. 2022. QR Code Reader. <https://apps.apple.com/us/app/qr-code-reader/id1200318119>. [Online; accessed 7-April-2022].
- [31] Ian Traynor. 2007. Russia accused of unleashing cyberwar to disable Estonia. <https://www.nytimes.com/2007/03/07/technology/russia-ukraine-internet-isolation.html>. [Online; accessed 17-May-2007].
- [32] Wikipedia. 2021. Data Matrix — Wikipedia, The Free Encyclopedia. <http://en.wikipedia.org/w/index.php?title=Data%20Matrix&oldid=1039532414>. [Online; accessed 10-September-2021].
- [33] Wikipedia. 2021. Encapsulated PostScript — Wikipedia, The Free Encyclopedia. <http://en.wikipedia.org/w/index.php?title=Encapsulated%20PostScript&oldid=1034253472>. [Online; accessed 10-September-2021].
- [34] Wikipedia. 2021. QR code — Wikipedia, The Free Encyclopedia. <http://en.wikipedia.org/w/index.php?title=QR%20code&oldid=1043292836>. [Online; accessed 10-September-2021].
- [35] Karl D. D. Willis and Andrew D. Wilson. 2013. InfraStructs: Fabricating Information inside Physical Objects for Imaging in the Terahertz Region. *ACM Trans. Graph.* 32, 4, Article 138 (July 2013), 10 pages. <https://doi.org/10.1145/2461912.2461936>
- [36] ZXing. 2021. ZXing. <https://github.com/zxing/zxing>. [Online; accessed 10-September-2021].